

Chapter 4

Software

Cambridge IGCSE & O Level Computer Science
(2210 / 0478)

Covers: 4.1 Types of software & interrupts
4.2 Programming languages, translators & IDEs

Made ready-to-learn with definitions, diagrams, exam tips & past-
paper style practice

4.1 Types of Software & Interrupts

4.1.1 System Software vs Application Software

All computers are just **hardware** until software gives them a job to do. Software sits in two big categories:

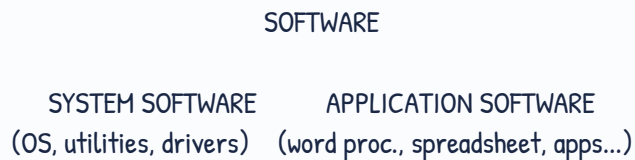


Fig — software family tree

System software: runs and manages the hardware itself + gives other software a platform to run on. Controls resources, provides the human-computer interface.

Application software: lets the USER do a specific task (typing a letter, editing a photo, browsing). It runs ON TOP of system software.

Exam trap — a compiler, device driver, and OS are all system software, even though a programmer "uses" a compiler — because its job is to manage/enable, not to do a user-task directly.

Common Application Software

Type	What it does
Word processor	Create/edit text docs — copy/paste, spellcheck, formatting
Spreadsheet	Numbers in a grid; formulas, graphs, "what if" modelling
Database	Stores data in tables (rows=records, columns=fields); queries & reports
Control/measuring s/w	Talks to sensors — measures & controls real-world processes
Photo / video / graphics editors	Manipulate images, video, bitmap or vector graphics
Apps	Mobile software from an app store (GPS, streaming, camera...)

Common System Software

Type	What it does
Operating System	Manages everything — the "boss" of the computer
Compiler / Linker	Translate & combine programmer's code into a runnable program
Device driver	Lets a hardware device "talk" to the OS
Utility programs	Maintain & manage resources (antivirus, backup, defrag...)

Utility Programs — Deep Dive

Utilities are system software that keep the computer healthy. Most just run quietly in the background.

Virus Checkers

- Scan files/software before they run or load
- Compare against a database of **known viruses**
- **Heuristic checking** — flags suspicious BEHAVIOUR, catches new/unknown viruses too
- Infected files go into **quarantine** (auto-delete OR user decides — risk of **false positive**)
- Must stay updated + run full scans weekly (some viruses lie dormant)

Defragmentation Software

Over time, files get split into scattered, non-contiguous sectors on an HDD slower access (read-write head jumps around). A defragmenter rearranges data into **contiguous sectors**.

Exam tip — SSDs do NOT need defragmenting (no moving read-write head, so no benefit).

Back-up Software

Best practice = 3 copies of a file:

1. Current working version (on internal HDD/SSD)
2. Local back-up (external drive)
3. Remote back-up (e.g. cloud storage)

Windows File History (hourly snapshots). Mac OS Time Machine (hourly / daily / weekly, oldest deleted when drive fills up).

Security Software

- Manages user accounts / access control (IDs + passwords)
- Works with firewalls, antivirus, anti-spyware
- Uses **encryption/decryption** so intercepted data is useless
- Checks software updates come from a legit source

Screensavers

Originally to stop CRT "phosphor burn" — not needed on modern LCD/OLED. Now mainly used for security (auto-lock after inactivity) and to trigger background tasks (virus scans) while idle.

Device Drivers

Translate data so a hardware device can communicate with the OS. Contain **descriptors** — vendor ID, product ID, unique serial number — so USB devices are recognised. No serial number OS treats it as "new" every time it's plugged in.

4.1.2 Operating Systems — The 9 Functions

Memory trick: "H.S.I.M.M.P.H.M.U" — just remember the 9 boxes below!

HCI | Security | Interrupt handling | Multitasking
Memory mgmt | Platform for apps | Hardware/peripheral mgmt
File management | User account management

Fig 4.7 — Operating System functions (all sit "around" the OS core)

1. Human Computer Interface (HCI)

CLI (Command Line)	GUI (Graphical)
Type commands directly	Click icons/windows (WIMP)
+ Direct control, not limited to menus, uses less memory	+ User-friendly, no commands to learn
— Must memorise commands, slow, error-prone typing	— Limited to icons given, uses more memory
Used by: programmers, technicians	Used by: everyday end-users

2. Memory Management

- Moves data between RAM HDD/SSD as programs run
- Tracks all memory locations
- **Memory protection** — stops 2 apps using the same memory location (avoids data loss, wrong results, security holes, crashes)

3. Security Management

OS updates, keeping antivirus current, working with firewalls, user privileges/accounts, access rights, data recovery/system restore.

4. Hardware / Peripheral Management

Talks to I/O devices via drivers; manages **queues & buffers**. Printer example: driver loaded data buffer (queue if busy) sent to printer handles errors/interrupts.

5. File Management

Naming conventions (filename.extension), create/open/delete/rename/move, directory structures, access control, loading files into memory.

6. Multitasking

Runs many processes "at once" by sharing hardware — each process gets a time limit, can be interrupted, and has a priority (low priority process risks resource starvation).

7. User Account Management

Each user gets their own protected space (username+password); admin can create/delete/restrict accounts; supports multi-access levels (e.g. clerk can order stationery but can't see HR files).

4.1.3 Running of Applications — Boot-up

Booting up: when the computer starts, part of the OS is loaded into RAM (also called a **bootstrap loader**).

- **BIOS** (Basic Input/Output System) starts the motherboard, finds the storage device holding the OS, loads & runs the needed part
- BIOS = **firmware** (low-level control program for hardware devices)
- BIOS program stored on **EEPROM** (survives power-off, but can be rewritten)
- BIOS settings stored on **CMOS** chip, kept alive by a small battery — remove battery settings reset to factory default

Exam trap — BIOS program (EEPROM) $\neq$ BIOS settings (CMOS). They use different memory for different reasons — examiners love this question!

4.1.4 Interrupts

Interrupt: a signal sent from hardware or software to the microprocessor telling it to temporarily pause its current task and deal with something else.

Caused by: a timer signal • an I/O device needing attention (disk/printer) • a hardware fault (paper jam) • user action (Ctrl+Alt+Break) • software errors (divide by zero, missing file, memory clash)

Buffers — the interrupt's best friend

Buffer: a temporary memory area that holds data while a slow device (like a printer) catches up with a fast processor.

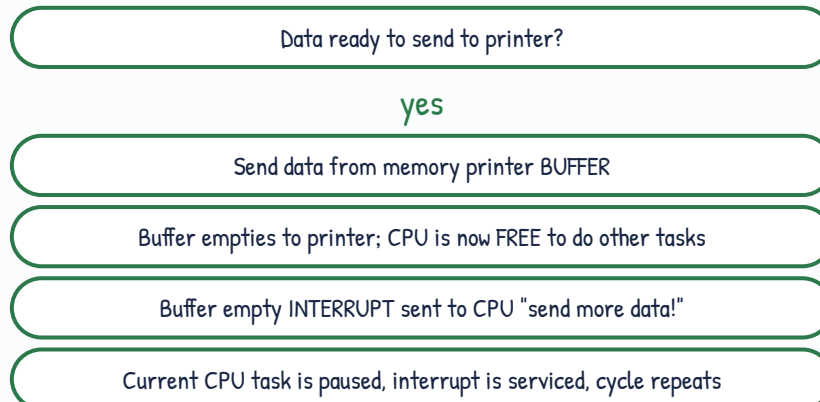


Fig 4.13 — how buffers + interrupts stop the CPU going idle while printing

Servicing an interrupt — the 3 steps

1. Save the status of the current task (Program Counter + registers)
2. Load start address of the **Interrupt Service Routine (ISR)** into the PC and run it
3. Once done, restore the saved registers and resume the original task

4.2 Programming Languages, Translators & IDEs

4.2.1 High-Level vs Low-Level Languages

High-level (easy to read):

```
Sum := FirstNumber + SecondNumber
```

Low-level / Machine code (hex):

```
12 13 1A
```

High-Level	Low-Level
+ Independent of computer type (portable)	+ Uses special hardware / machine-specific instructions
+ Easier to read/write/understand (closer to English)	+ Small memory footprint
+ Quicker to write, debug & maintain	+ Executes very fast for specific tasks
– Larger programs, may run slower	– Hard to understand, longer to write/debug
– May not access special hardware directly	– Machine-dependent (not portable)

Examples of HLL: Python, Java, C++, Visual Basic, Pascal. LLL = machine code (binary) or assembly language.

4.2.2 Assembly Language

Programmers use assembly for: special hardware access, machine-specific instructions, tiny memory footprint, or raw speed.

```
LDA First    // load First into accumulator
ADD Second  // add Second to accumulator
STO Sum     // store result in Sum
```

Assembly language must be translated into machine code by an **assembler** before it can run.

4.2.3 Translators — Compare the 3

Compiler	Interpreter	Assembler
Translates whole HLL program machine code, all at once	Translates & runs ONE HLL statement at a time	Translates a low-level assembly program machine code
Produces an executable file	No executable file produced	Produces an executable file
Runs without the compiler afterwards	Cannot run without the interpreter	Runs without the assembler afterwards
1 HLL statement several machine instructions	1 statement may need several instructions each run	Usually 1 assembly line 1 machine instruction
Used for final/distributed programs	Used during development	Used for final/distributed programs

4.2.4 Compiler vs Interpreter — Pros & Cons

Compiler	Interpreter
+ Compiled program runs without needing the compiler again	+ Easier & quicker to debug/test WHILE developing
+ Takes up less memory when executing	+ Easier to edit programs during development
+ Executes in a shorter time overall	— Program can't run without the interpreter present
— Takes longer to write/test/debug during development	— Slower execution (translates every single run)

Exam tip — "why interpreter during development?" because you get INSTANT feedback per line, so bugs are found fast, without waiting for the whole program to compile.

4.2.5 IDE — Integrated Development Environment

Examples: PyCharm (Python), Visual Studio (VB), BlueJ (Java).

Feature	What it does for the programmer
Code editor	Write & edit code in one place — no separate text editor needed
Translator	Built-in compiler and/or interpreter to run the code
Runtime + Debugger	Step through code line-by-line or set breakpoints ; inspect variable values to catch logic errors
Error diagnostics	Dynamic checking as you type — underlines errors + suggests fixes
Auto-completion	Suggests variable names / reserved words as you type
Auto-documenter	Auto-generates explanation of code's purpose
Prettyprinting	Colour-codes & neatly formats/indents code automatically

Past-Paper Style Practice

Questions written in the Cambridge exam style for self-practice, with a model marking scheme below each one.

Q1

[3]

State three functions of an operating system.

Mark scheme (1 mark each, max 3):

- Memory management
- Security management / user account management
- Providing a human-computer interface
- Managing files / hardware peripherals / multitasking (any 3 from list)

Q2

[4]

Describe how buffers and interrupts work together when a document is printed.

Mark scheme (1 mark per valid point, max 4):

- Data is sent from memory to a printer buffer
- CPU is freed to carry out other tasks while buffer empties
- When buffer is empty, an interrupt is sent to the processor
- Interrupt requests more data / current task is paused to service it

Q3

[2]

Explain why a programmer might choose to write a program in assembly language rather than a high-level language.

Mark scheme (1 mark each, max 2):

- To access special/specific hardware directly
- To write code that uses very little memory / runs very fast

Q4

[3]

A student says "a compiler and an interpreter do the same job." Explain why this statement is not fully correct.

Mark scheme (max 3):

- A compiler translates the whole program at once into an executable file
- An interpreter translates and runs one statement at a time, no executable is produced
- A compiled program can run without the compiler; an interpreted one always needs the interpreter present

Q5

[2]

Describe the difference between the BIOS program and the BIOS settings, in terms of the memory used to store each.

Mark scheme (max 2):

- BIOS program is stored on EEPROM (retains data without power, can be updated)
- BIOS settings are stored on CMOS (kept alive by a battery; resets to default if battery removed)

Before your exam — Practice drawing the buffer/interrupt printer flowchart from memory, and re-do the CLI vs GUI table without looking. These two come up almost every year.